UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Web Applications A.Y. 2025-2026

Homework 1 – Server-side Design and Development

Master Degree in Computer Engineering
Master Degree in Cybersecurity
Master Degree in ICT for Internet and Multimedia

Deadline: 17 April, 2026

Group Acronym	WASP	
Last Name	**First Name**	**Badge Number**
Bardhi	Rajmonda	2071810
Karriqi	Meviola	2183853
Perin	Alessandro	2203653
Smirnov	Danil	2142049
Masiero	Giacomo	2130974

1 Objectives

FinTrack is a personal finance management web application designed to help individuals monitor, analyse, and plan their financial situation through a single, unified platform.

The primary objective of the system is to give users full visibility over their financial state. Many people manage money across multiple accounts or wallets without a consolidated view of their overall position. FinTrack addresses this by aggregating all wallets and transactions into a single dashboard that shows total equity, year-to-date expenses, and year-to-date savings at a glance.

A second objective is to support forward-looking financial planning. Knowing the current balance is not enough if a user also wants to understand how their finances may evolve over time. FinTrack therefore provides a forecasting module that projects income, expenses, PAC contributions, and net balance month by month over a 24-month horizon under three scenarios: optimistic, conservative, and stress test. This allows users to anticipate potential shortfalls and make informed decisions before problems arise.

A third objective is to encourage disciplined long-term saving through PAC plans. A PAC (Piano di Accumulo Capitale) is a recurring investment plan in which a fixed amount is contributed each month toward a defined target. FinTrack lets users create, track, and adjust these plans, and integrates them directly into the forecasting logic so that their impact on future balances is always visible.

Finally, the system aims to be accessible even to users who are not yet registered. A public guest area provides three standalone financial calculators — loan amortisation, compound interest, and budget analysis — so that anyone can use the core analytical tools without creating an account. This lowers the barrier to entry and demonstrates the value of the platform before a commitment is made.

2 Main Functionalities

- *Registration and Login:* New users must register by providing their name, surname, email address, and password. Passwords are hashed using PBKDF2 (pure Java `javax.crypto`) before storage and are never persisted in plain text. Once registered, users can log in to authenticate their session and access all protected features. Failed login attempts return an error message without exposing internal details.
- *Guest Access:* Users who do not wish to create an account can continue as guests. A session-scoped UUID token is generated and persisted in the `report_sessions` table, allowing guests to generate one-off financial reports without registration.
- *Wallet Management:* Authenticated users can create, view, edit, and delete financial wallets. Each wallet has a name, type (cash, bank account, credit card, or investment), and a 3-letter currency code. All wallets under one account must use the same currency. Deletion is rejected if the wallet contains linked transactions.
- *Transaction Tracking:* Users can record income and expense transactions within their wallets. Transactions are managed through a REST/JSON API, allowing interactive creation, editing, and deletion without full page reloads. Positive amounts are treated as income and negative amounts as expenses, resolved automatically at the service layer. Amounts are stored as absolute values with two decimal places.
- *PAC (DCA) Investment Plans:* Users can create and manage Dollar Cost Averaging investment plans. Each plan specifies an asset name, a monthly contribution amount, a target amount, and a start date. Plans can be activated or deactivated and are factored into the forecasting engine.
- *Recurring Expenses:* Users can define recurring financial obligations with configurable periodicity (monthly, quarterly, or yearly). Active recurring expenses are normalized to monthly equivalents and included in forecast projections.
- *Financial Forecasting:* The application provides a 24-month net worth projection engine accessible via a REST/JSON API. Users select one of three scenarios:
  - **Optimistic:** income increased by 10%, expenses unchanged.

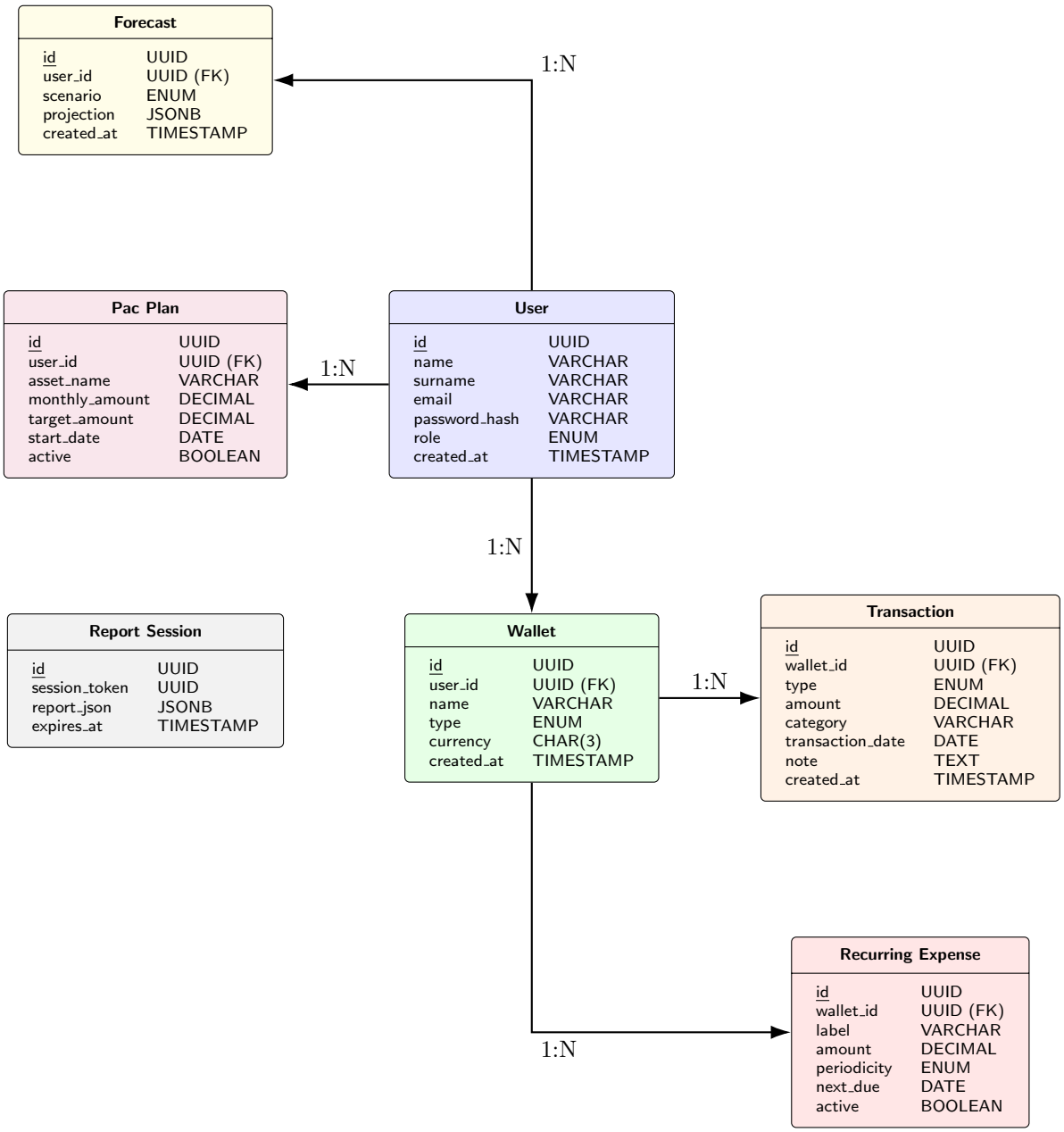
- **Conservative:** income unchanged, expenses subject to 3% compounding monthly inflation.
- **Stress Test:** income reduced by 20%, with a one-off expense shock of 3× the average monthly expense at month 6.

Projections are computed from current wallet balances, the 3-month average of income and expenses, active recurring expenses, and active PAC contributions. Results are stored as JSONB in the database for later retrieval.

- *Dashboard:* After logging in, users are redirected to a dashboard that serves as the central hub for navigating wallets, transactions, PAC plans, and forecasts.

3 Data Logic Layer

3.1 Entity-Relationship Schema



The application's data is persisted in a relational database mapping its core entities. The ER schema models the users, their financial wallets, atomic exchanges (transactions), recurring events, saving plans, analytical forecasts, and temporary sessions.

The primary attributes and relationships of each entity are defined as follows:

- **User**: Represents the central entity, storing personal details, authentication credentials (via `password_hash`), and the assigned `role` (e.g., `REGISTERED` or `GUEST_SESSION`).
- **Wallet**: A one-to-many relationship exists between User and Wallet since a user can possess multiple wallets

(e.g., Cash, Bank Account). Each wallet maintains a strict `user_id` reference (with cascading deletion) and stores the `currency` used.

- **Transaction**: Linked to a specific Wallet (`wallet_id`), representing an atomic financial entry (`INCOME` or `EXPENSE`). It stores crucial financial data such as `amount`, `category`, `transaction_date`, and an optional `note`.
- **Recurring Expense**: Also associated with a Wallet, this entity defines scheduled cash outflows, keeping track of the `periodicity` (e.g., Monthly), the `next_due` date, and an `active` flag for toggling state without deletion.
- **Pac Plan**: Defines periodic investment (PAC) goals mapped directly to a User. It captures the `asset_name`, `monthly_amount`, and the overall `target_amount`.
- **Forecast**: A purely analytical entity connected to a User, storing a chosen scenario (e.g., `OPTIMISTIC`) and the resultant JSON structural breakdown of the `projection_json`.
- **Report Session**: A transient entity used for temporary guest access, managing a `session_token`, the embedded `report_json`, and an `expires_at` timestamp.

3.2 Other Information

In addition to the standard conceptual schema, the Data Logic Layer incorporates several implementation-specific features at the database level to ensure data integrity and operational safety:

- **Primary Keys and Security**: All entities utilize universally unique identifiers (`UUIDv4`) as primary keys instead of predictable sequential integers. This choice mitigates enumeration attacks and aids in distributed data handling across services.
- **Strong Typing with PostgreSQL ENUMs**: Business constraints on categorical fields are strictly enforced at the database level using custom ENUM types (e.g., `wallet_type`, `transaction_type`, `periodicity_type`, `forecast_scenario`). This natively prevents invalid states rather than relying solely on the application layer validation.
- **JSONB Serialization**: Elements with dynamic structural logic, such as forecast projections (`projection_json`) and guest report sessions (`report_json`), leverage PostgreSQL's `JSONB` binary format. This allows high-performance reads and flexible data structures for analytical features without rigid schema changes.
- **Defensive Constraints**: Critical numerical fields (such as `amount` in transactions and `monthly_amount` in plans) are safeguarded with explicit `CHECK` constraints directly in the DDL to enforce strictly positive values.
- **Data Lifecycle**: Most relational links (e.g., wallet to user, transaction to wallet) enforce `ON DELETE CASCADE` constraints. This guarantees that when a parent entity is purged (like a user account), all subordinate financial data is automatically expunged, effectively preventing orphaned records.

4 Presentation Logic Layer

The presentation layer of FinTrack is organized around a public entry flow, an authenticated area for registered users, and a guest-only calculator area. The main pages are the following:

- **Home page:** The landing page of the application, where users can choose whether to log in, register, or continue as guest.
- **Login page:** A page for returning users, with an email and password form and error feedback for invalid credentials or missing input.
- **Registration page:** A page for new users, with a form for name, surname, email, and password, plus validation and duplicate-email feedback.
- **Dashboard page:** The main authenticated overview page, showing the user's financial summary and the list of wallets currently associated with the account.
- **Forecasting page:** An authenticated analytical page dedicated to scenario-based financial projections generated from the user's transactions, recurring expenses, and PAC plans.
- **PAC page:** A protected page where the user can create, inspect, update, and delete PAC plans.
- **Guest page:** A public calculator area that allows non-registered users to access the loan, compound-interest, and budget calculators together with their related reports.

4.1 Home Page

The Home page is the public landing page of FinTrack. Its purpose is to present the three main entry points of the system in a simple and immediate way. From this page, the user can choose to authenticate through the Login page, create a new account through the Registration page, or continue without registration by entering the guest calculator area. The page therefore acts as the starting point of the whole navigation flow. It is intentionally lightweight, because its role is not to show financial data but to route the visitor toward the correct experience depending on whether the visitor is already registered or only wants to use the public calculators.

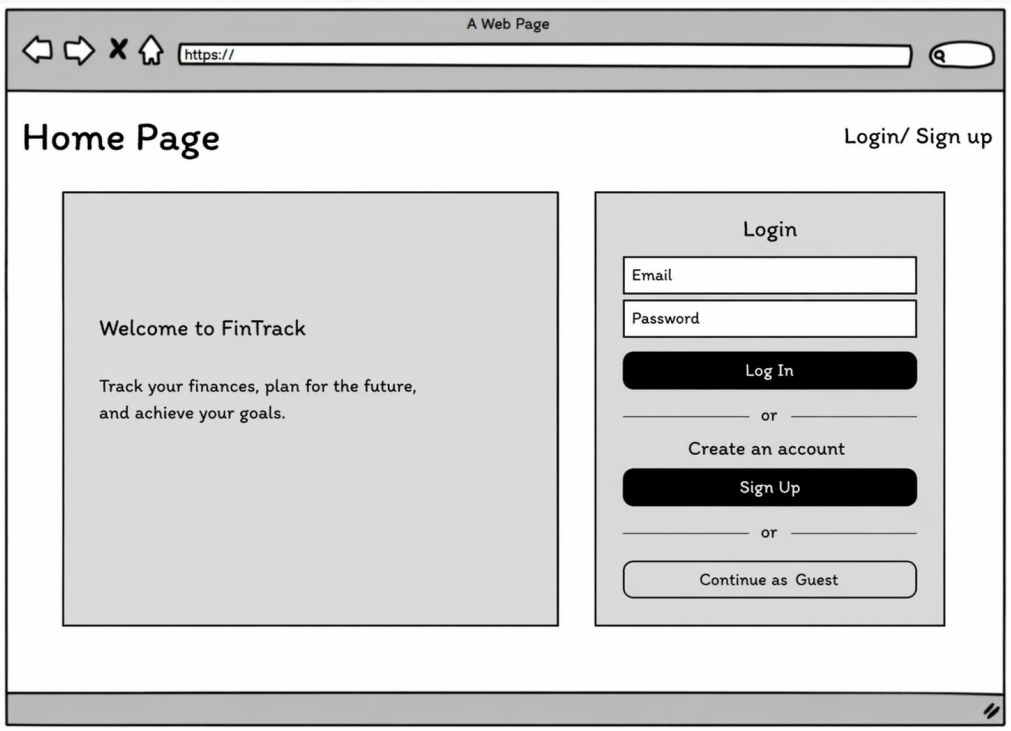


Figure 1: Home Page

4.2 Login Page

The Login page is designed for returning users who want to access the protected area of the application. The interface contains a form with two input fields, one for the email address and one for the password. When the form is submitted, the credentials are sent to the authentication servlet, which validates the input and checks whether the account exists and the password is correct. If the authentication succeeds, the user session is initialized and the user is redirected to the Dashboard page. If the input is incomplete or the credentials are invalid, an error message is shown on the same page. A link to the Registration page is also provided so that a visitor without an account can move directly to the sign-up flow.

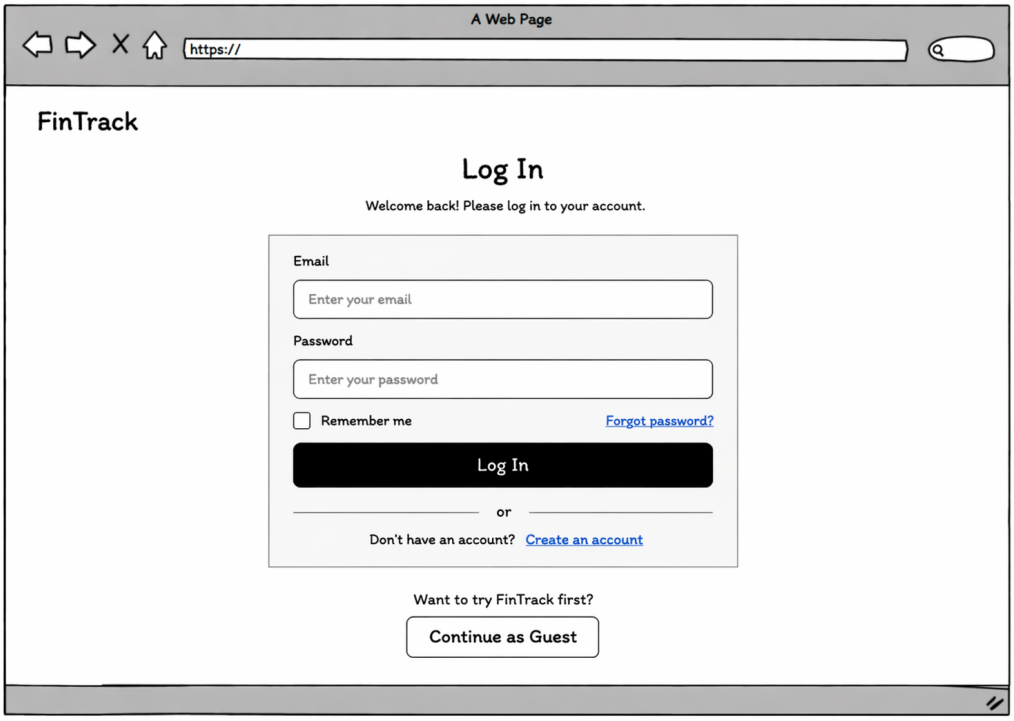


Figure 2: Login Page

4.3 Registration Page

The Registration page allows a new user to create a personal FinTrack account. The page contains a form with the fields required by the application logic: name, surname, email, and password. These values are validated before the account is created, so the page must support the display of feedback messages in case one or more fields are missing or the email address is already associated with another account. After a successful registration, the application immediately opens an authenticated session for the new user and redirects the user to the Dashboard page. In this way, the registration flow is integrated with the main personal area of the system without requiring a second login step. The page also includes a link back to the Login page for users who already have an account.

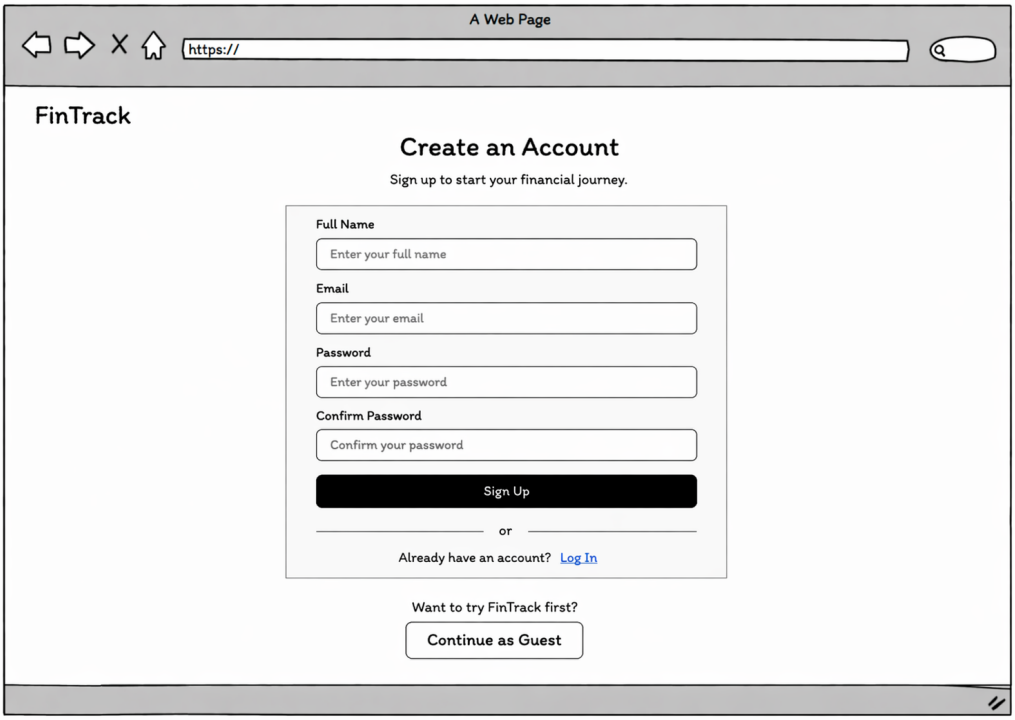


Figure 3: Registration Page

4.4 Dashboard Page

The Dashboard page is the main overview page for authenticated users. At the top, it greets the user by name, confirming that the personal session is active. The page then shows a financial summary containing the main high-level indicators currently produced by the business logic: total equity, year-to-date expenses, and year-to-date savings. Below the summary, the page displays the list of wallets owned by the user. For each wallet, the interface shows the wallet name, wallet type, currency, and current balance. If the user has not created any wallet yet, the page displays a message inviting the user to create the first one. A logout button is also provided so that the user can explicitly terminate the authenticated session.

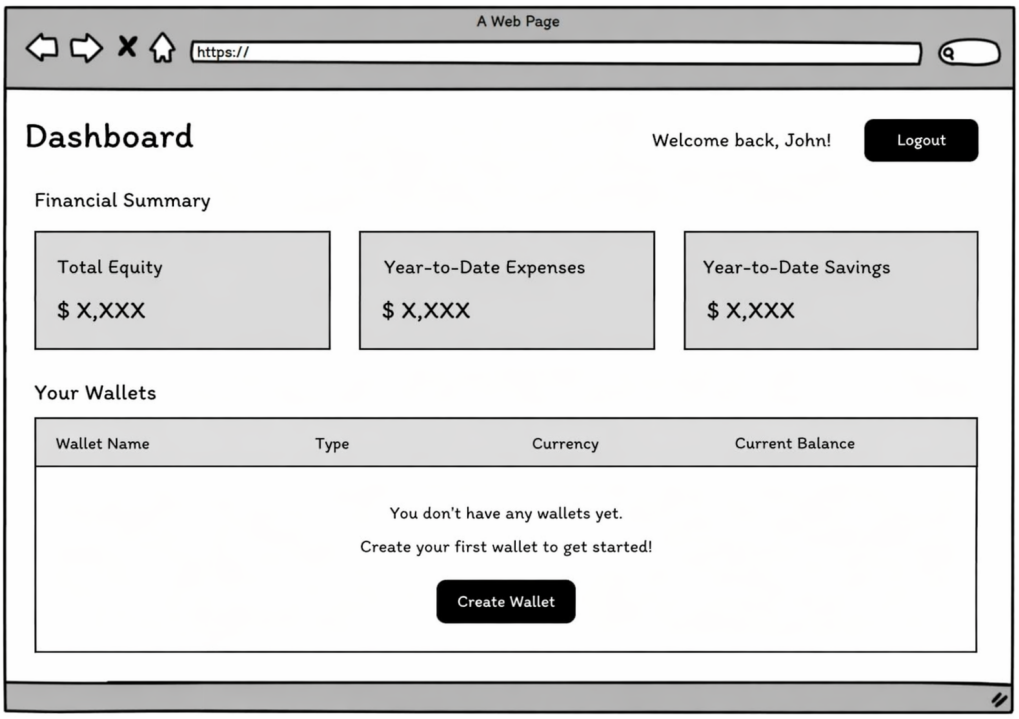


Figure 4: Dashboard Page

4.5 Forecasting Page

The Forecasting page is an authenticated analytical page focused on future financial projections. In the current source code, the forecasting functionality is backed by the `/api/forecast` endpoint, which accepts a scenario and returns a generated projection for the authenticated user. The supported scenarios are `OPTIMISTIC`, `CONSERVATIVE`, and `STRESS_TEST`. From a presentation point of view, the page is meant to let the user choose one of these scenarios and inspect the generated projection. Each forecast is computed over 24 months and includes, for every month, the estimated income, expense, PAC contribution, net flow, and resulting balance. The page therefore provides a visual and tabular summary of how the user's finances may evolve under different assumptions.

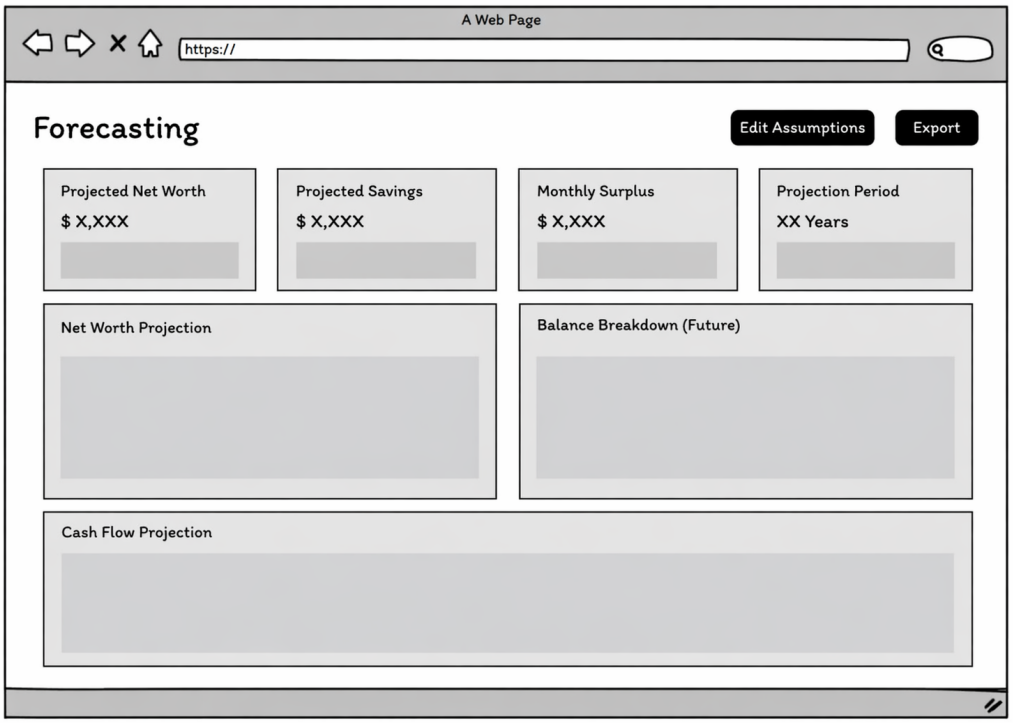


Figure 5: Forecasting Page

4.6 PAC Page

The PAC page is a protected management page dedicated to recurring investment plans. According to the application logic, each PAC plan contains an asset name, a monthly contribution amount, a target amount, a start date, and an active flag. The page must therefore allow the user to visualize the list of plans already stored in the system and to perform the main CRUD operations on them. In particular, the interface supports plan creation, modification, and deletion through the dedicated PAC servlets. The page also shows success or error messages after each operation, for example when a plan is created correctly, when validation fails, or when the user attempts an unauthorized action. This page is important because PAC plans are also reused by the forecasting logic as part of the monthly projection.

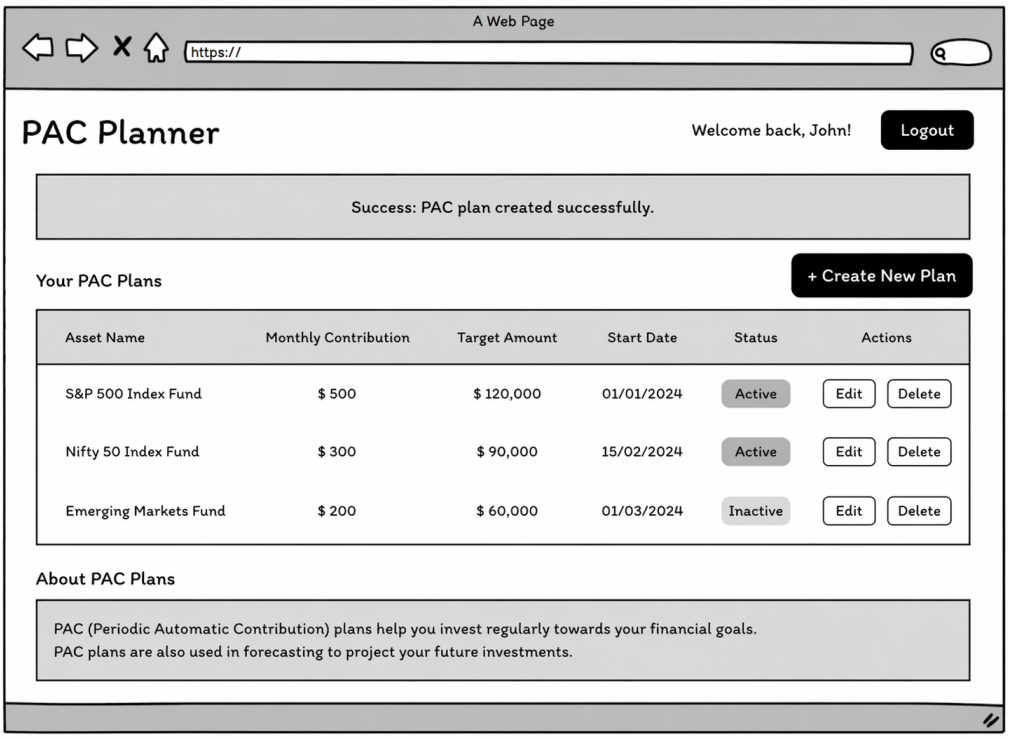


Figure 6: PAC Page

4.7 Guest Page

The Guest page is the public calculator area of FinTrack and is accessible without creating an account. When a user enters this area, the system associates the browser session with a guest token and forwards the request to the calculator page. The interface groups together three independent tools: a loan calculator, a compound-interest calculator, and a budget calculator. The loan calculator asks for principal, annual rate, and duration in months, then produces a report page showing the amortization table month by month. The compound-interest calculator collects the initial capital, monthly contribution, annual rate, and duration in years, then shows the total value reached year by year. The budget calculator lets the user dynamically add and remove income and expense rows, and after submission it generates a report page with total income, total expenses, net balance, and a category-by-category breakdown. The Guest page therefore offers a complete public entry point for trying the financial analysis tools before registration.

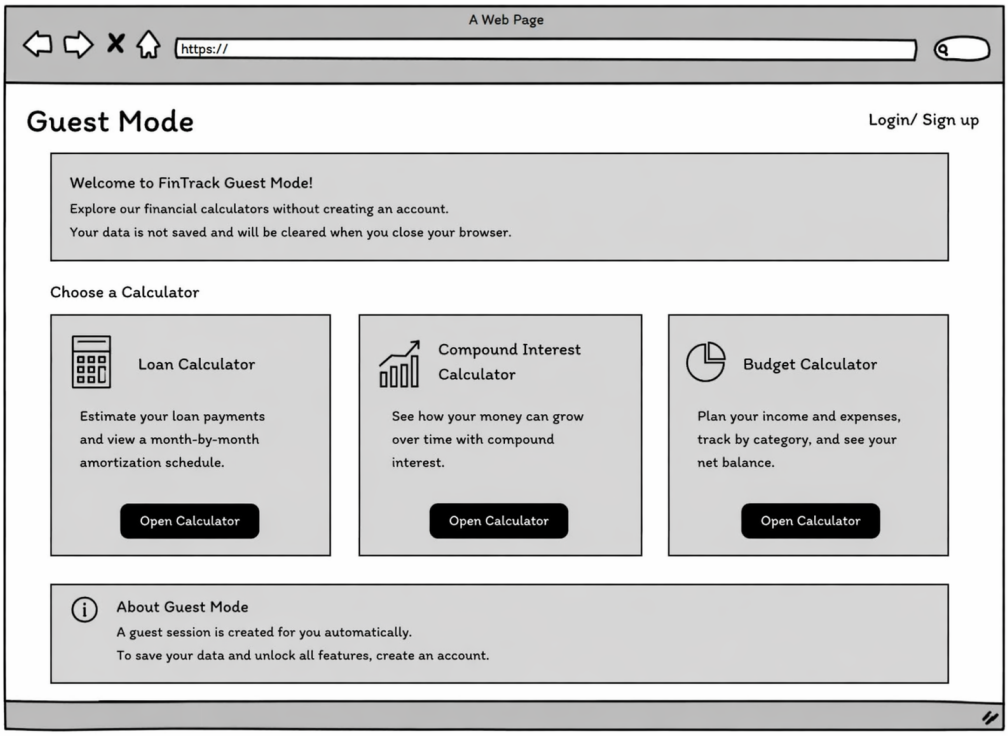


Figure 7: Guest Page

5 Business Logic Layer

5.1 Class Diagram

Add `ClassDiagram.pdf` here.

The business logic layer of FinTrack is organized around different services classes, each responsible for a distinct functional area of the application. All service classes follow a consistent pattern: they accept validated input, apply business rules, and delegate persistence operations to the corresponding DAO class in the data layer. On this chapter, we are going to illustrate the structure of classes, and understand his functionality:

- **UserService** class handles user registration and authentication. During registration, it normalizes and validates all input fields, checks for duplicate emails via `UserDAO`, and delegates password hashing to `PasswordUtil`. On the other hand, during login it retrieves the stored hash and verifies the input password using the same utility. Both operations raise typed exceptions (`ValidationException`, `InvalidCredentialsException`, `UserAlreadyExistsException`) that are caught at the servlet level and displayed to the user.
- **WalletService** class manages the lifecycle of user wallets. It enforces two invariants: first, all field values are validated (non-blank name, valid 3-letter alphabetic currency code, recognized wallet type); second, all wallets belonging to the same user must share a single currency — a constraint checked at creation and update time via `SingleCurrencyRequiredException`. Ownership is verified before any mutation, and deletion is blocked if the wallet contains linked transactions (`WalletHasTransactionsException`).
- **TransactionService** class handles creation, update, and deletion of financial transactions. It determines the transaction type (INCOME or EXPENSE) from the sign of the amount, stores the absolute value, and verifies wallet ownership before any write operation.
- **RecurringExpenseService** class manages recurring fixed costs associated with a wallet. It supports creation, update, deletion, and toggling of the active status.
- **PacPlanService** class handles the PAC (Piano di Accumulo Capitale) investment plans. Each plan stores a monthly contribution amount and a target amount; the service validates all numeric fields and verifies ownership before mutations.
- **DashboardService** class use the `DashboardDAO` to obtain two parallel result sets — per-wallet balance summaries and per-currency YTD statistics — and wraps them in a `DashboardSummary` value object consumed by the presentation layer.
- **ForecastService** class produce a 24-month financial projection for a given user, and it does so by combining several sources of financial data into a single estimate. First, it reconstructs the user's current balance by summing all historical transactions. It then calculates the average monthly income and expenditure over the most recent three months, using a shorter window when the user does not yet have enough history. Active recurring expenses are also taken into account: each one is normalized to a monthly equivalent depending on its frequency, so a quarterly charge is divided by three and a yearly one by twelve. Finally, any active PAC plan contributions are added to the monthly outflow.

  Once these figures are assembled, the service projects them forward month by month under one of three user-selected scenarios. In the `OPTIMISTIC` scenario, monthly income is increased by 10%. In the `CONSERVATIVE` scenario, expenses grow each month according to a 3% annual inflation rate. In the `STRESS_TEST` scenario, income is reduced to 80% of its average and a one-time expense shock equal to three times the average monthly expenditure hits at month six, simulating an unexpected financial event. A prerequisite for the computation is that all of the user's wallets share the same currency; if they do not, a `SingleCurrencyRequiredException` is raised and the computation is aborted. The final projection is serialized as a JSON array and stored in the database via `ForecastDAO` for later retrieval.

- **CalculatorService** class is a stateless service used exclusively in the guest area. It provides three independent calculators: loan amortization (using the standard annuity formula, with a degenerate zero-rate case), compound interest (monthly compounding with optional monthly contributions), and budget analysis (aggregating income and expense categories into totals and a net balance). All inputs are validated and financial values are computed using `BigDecimal` at 2 decimal places.
- **GuestSessionService** class manages the lifecycle of temporary guest sessions. When a guest visits a calculator endpoint, the service checks whether a valid, non-expired `ReportSession` already exists for the token stored in the HTTP session. If not (or if the session has expired), a new UUID is generated, a new `report_sessions` row with a 24-hour TTL is created via `ReportSessionDAO`, and the new token is returned to the filter for storage in the HTTP session.
- **PdfReportService** class generates downloadable PDF files using Apache PDFBox. For guest users, it loads the accumulated JSON from the `ReportSession`, validates that it is non-empty and not expired, and renders up to three sections (loan schedule, compound interest projection, budget breakdown). For authenticated users, it fetches live data from `DashboardDAO`, `RecurringExpenseDAO`, and `ForecastDAO` and renders four sections (wallet balances, YTD summary, recurring expenses, and the forecast projection).

5.2 Sequence Diagram

Add `sequenceDiagram.pdf` here.

5.3 REST API Summary

The application exposes two categories of endpoints: Servlet/JSP endpoints using the Post/Redirect/Get pattern for page-based interactions, and REST/JSON endpoints for AJAX-driven features. Both categories are listed below.

Servlet/JSP Endpoints (PRG Pattern)

URI	Method	Description	Filter
`/`	GET	Home page.	–
`/login`	GET	Display login page.	–
`/auth/login`	POST	Authenticate user; redirect to `/dashboard`.	–
`/register`	GET	Display registration page.	–
`/auth/register`	POST	Create new user; redirect to `/dashboard`.	–
`/auth/logout`	POST/GET	Invalidate session; redirect to `/login`.	–
`/dashboard`	GET	Dashboard with financial summary.	Auth
`/wallets`	GET	List all wallets for the user.	Auth
`/wallets/create`	POST	Create a new wallet; redirect to `/wallets`.	Auth
`/wallets/edit`	POST	Update wallet; redirect to `/wallets`.	Auth
`/wallets/delete`	POST	Delete wallet; redirect to `/wallets`.	Auth
`/pac`	GET	List all PAC plans for the user.	Auth
`/pac/create`	POST	Create a new PAC plan; redirect to `/pac`.	Auth
`/pac/edit`	POST	Update PAC plan; redirect to `/pac`.	Auth
`/pac/delete`	POST	Delete PAC plan; redirect to `/pac`.	Auth
`/guest/report`	GET	Guest calculator page.	Guest
`/report/loan`	POST	Compute loan amortization schedule.	Guest
`/report/compound`	POST	Compute compound interest projection.	Guest

/report/budget	POST	Compute budget breakdown.	Guest
/report/export	GET	Export report as PDF.	–

Table 2: Servlet/JSP endpoints. "Auth" = AuthenticationFilter (redirects to /login). "Guest" = GuestSessionFilter (initializes guest token).

REST/JSON API Endpoints

URI	Method	Description	Filter
/api/transactions?walletId={walletId}	GET	List all transactions for a wallet.	Auth
/api/transactions	POST	Create a new transaction.	Auth
/api/transactions/{transactionId}	PUT	Update a transaction.	Auth
/api/transactions/{transactionId}	DELETE	Delete a transaction.	Auth
/api/expenses	GET	List all recurring expenses for the user.	Auth
/api/expenses	POST	Create a new recurring expense.	Auth
/api/expenses/{expenseId}	PUT	Update a recurring expense.	Auth
/api/expenses/{expenseId}	DELETE	Delete a recurring expense.	Auth
/api/expenses/{expenseId}/toggle	POST	Toggle active/inactive status.	Auth
/api/forecast?scenario={scenario}	POST	Compute 24-month projection.	Auth
/api/forecast?scenario={scenario}	GET	Retrieve latest stored projection.	Auth

Table 3: REST/JSON API endpoints. All /api/* routes are protected by AuthenticationFilter, which returns a JSON 401 response for unauthenticated requests instead of redirecting.

5.4 REST Error Codes

All REST API endpoints return errors as JSON objects in the format {"error": "message"}. The following table summarizes the HTTP status codes used across the API and their meaning.

Error Code	HTTP Status	Description
400	Bad Request	Malformed JSON body, missing required fields, invalid UUID format, invalid amount or date format, empty request body, oversized request body (¿8KB), invalid scenario value, or blank required parameters.
401	Unauthorized	Authentication required. Returned by AuthenticationFilter for unauthenticated /api/* requests, or by the servlet-level defensive check if the session has expired.
403	Forbidden	The authenticated user does not own the requested resource (e.g., attempting to modify another user's wallet or transaction).
404	Not Found	The requested resource does not exist (e.g., no forecast computed yet for the given scenario).

500	Internal Server Error	An unexpected server error occurred. Internal details are never exposed; the response contains only a generic error message.

Table 4: HTTP status codes used by the REST API.

5.5 REST API Details

This section documents the REST/JSON API endpoints and the Servlet/JSP endpoints. REST endpoints return JSON responses with appropriate HTTP status codes. Servlet/JSP endpoints follow the Post/Redirect/Get pattern and forward to JSP pages for rendering.

5.5.1 User Registration

Registers a new user. The password is hashed using PBKDF2 before storage.

- **URL:** `/auth/register`
- **Method:** `POST`
- **Data Parameters (form):**
  - `name` = {string}
  - `surname` = {string}
  - `email` = {string}
  - `password` = {string}
- **Success Response:**
  - HTTP 302 redirect to `/dashboard`; session created with authenticated user.
- **Error Responses:**
  - Redirect to `/register?error=email_exists` if the email is already registered.
  - Redirect to `/register?error=validation` if required fields are missing.
  - Redirect to `/register?error=server_error` on internal failure.

5.5.2 User Login

Authenticates an existing user by verifying email and password against the stored hash.

- **URL:** `/auth/login`
- **Method:** `POST`
- **Data Parameters (form):**
  - `email` = {string}
  - `password` = {string}
- **Success Response:**
  - HTTP 302 redirect to `/dashboard`; session created with authenticated user.

- **Error Responses:**
  - Redirect to `/login?error=invalid_credentials` if credentials are wrong.
  - Redirect to `/login?error=validation` if fields are missing.
  - Redirect to `/login?error=server_error` on internal failure.

5.5.3 User Logout

Invalidates the current session and redirects to the login page.

- **URL:** `/auth/logout`
- **Method:** `POST` or `GET`
- **Data Parameters:** None
- **Success Response:**
  - HTTP 302 redirect to `/login`.

5.5.4 List Wallets

Returns all wallets belonging to the authenticated user, forwarded to a JSP page for rendering.

- **URL:** `/wallets`
- **Method:** `GET`
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with the wallet list page rendered via JSP.
- **Error Responses:**
  - Redirect to `/login` if unauthenticated.

5.5.5 Create Wallet

Creates a new wallet for the authenticated user. Currency is normalized to uppercase and validated as a 3-letter alphabetic code. All wallets under one account must use the same currency.

- **URL:** `/wallets/create`
- **Method:** `POST`
- **Data Parameters (form):**
  - `name` = {string}
  - `type` = {string} (CASH, BANK_ACCOUNT, CREDIT_CARD, or INVESTMENT)
  - `currency` = {string} (3-letter code, e.g., EUR)
- **Success Response:**
  - Redirect to `/wallets?success=created`.

- **Error Responses:**
  - Redirect to `/wallets?error=validation` if fields are invalid.
  - Redirect to `/wallets?error=currency_mismatch` if currency differs from existing wallets.
  - Redirect to `/wallets?error=server_error` on internal failure.

5.5.6 Edit Wallet

Updates wallet name, type, or currency. Ownership is verified before mutation.

- **URL:** `/wallets/edit`
- **Method:** `POST`
- **Data Parameters (form):**
  - `walletId` = {UUID}
  - `name` = {string}
  - `type` = {string}
  - `currency` = {string}
- **Success Response:**
  - Redirect to `/wallets?success=updated`.
- **Error Responses:**
  - Redirect to `/wallets?error=validation` for invalid input or malformed UUID.
  - Redirect to `/wallets?error=not_found` if wallet does not exist.
  - Redirect to `/wallets?error=currency_mismatch` if currency differs from other wallets.
  - Redirect to `/wallets?error=unauthorized` if wallet belongs to another user.

5.5.7 Delete Wallet

Deletes a wallet. Rejected if the wallet contains linked transactions.

- **URL:** `/wallets/delete`
- **Method:** `POST`
- **Data Parameters (form):**
  - `walletId` = {UUID}
- **Success Response:**
  - Redirect to `/wallets?success=deleted`.
- **Error Responses:**
  - Redirect to `/wallets?error=validation` for missing or malformed UUID.
  - Redirect to `/wallets?error=not_found` if wallet does not exist.
  - Redirect to `/wallets?error=has_transactions` if wallet has linked transactions.
  - Redirect to `/wallets?error=unauthorized` if wallet belongs to another user.

5.5.8 List Transactions

Retrieves all transactions for a given wallet as JSON, ordered by transaction date descending. Wallet ownership is verified.

- **URL:** `/api/transactions?walletId={walletId}`
- **Method:** `GET`
- **URL Parameters:**
  - `walletId` = {UUID} (required)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with a JSON array of transaction objects.
- **Error Responses:**
  - HTTP 400 if `walletId` is missing or has an invalid UUID format.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the wallet does not belong to the authenticated user.

5.5.9 Create Transaction

Creates a new transaction within a wallet. The transaction type (income or expense) is resolved automatically from the sign of the amount: positive values are treated as income, negative values as expenses. The amount is stored as an absolute value scaled to two decimal places.

- **URL:** `/api/transactions`
- **Method:** `POST`
- **Data Parameters (JSON body):**
  - `walletId` = {UUID} (required)
  - `amount` = {string} (required, non-zero numeric value)
  - `category` = {string} (required, max 80 characters)
  - `date` = {string} (required, format YYYY-MM-DD)
  - `note` = {string} (optional, blank values normalized to null)
- **Success Response:**
  - HTTP 201 with the created transaction as a JSON object.
- **Error Responses:**
  - HTTP 400 if the request body is malformed, empty, oversized (¿8KB), or contains invalid field values.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the wallet does not belong to the authenticated user.

5.5.10 Update Transaction

Updates an existing transaction. Ownership is verified through the transaction's parent wallet.

- **URL:** `/api/transactions/{transactionId}`
- **Method:** `PUT`
- **URL Parameters:**
  - `transactionId` in path (UUID)
- **Data Parameters (JSON body):**
  - `amount` = {string} (required)
  - `category` = {string} (required)
  - `date` = {string} (required, format YYYY-MM-DD)
  - `note` = {string} (optional)
- **Success Response:**
  - HTTP 200 with the updated transaction as a JSON object.
- **Error Responses:**
  - HTTP 400 if the transaction ID is invalid, the body is malformed, or field values are invalid.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the transaction's wallet does not belong to the authenticated user.

5.5.11 Delete Transaction

Deletes an existing transaction. Ownership is verified through the transaction's parent wallet.

- **URL:** `/api/transactions/{transactionId}`
- **Method:** `DELETE`
- **URL Parameters:**
  - `transactionId` in path (UUID)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 204 No Content.
- **Error Responses:**
  - HTTP 400 if the transaction ID is missing or has an invalid format.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the transaction's wallet does not belong to the authenticated user.

5.5.12 List Recurring Expenses

Retrieves all recurring expenses for the authenticated user as JSON.

- **URL:** `/api/expenses`
- **Method:** `GET`
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with a JSON array of recurring expense objects.
- **Error Responses:**
  - HTTP 401 if the user is not authenticated.
  - HTTP 500 on internal failure.

5.5.13 Create Recurring Expense

Creates a new recurring expense linked to a wallet owned by the authenticated user.

- **URL:** `/api/expenses`
- **Method:** `POST`
- **Data Parameters (JSON body):**
  - `walletId` = {UUID} (required)
  - `label` = {string} (required)
  - `amount` = {string} (required, positive numeric value)
  - `periodicity` = {string} (required: MONTHLY, QUARTERLY, or YEARLY)
  - `nextDue` = {string} (required, format YYYY-MM-DD)
- **Success Response:**
  - HTTP 201 with the created recurring expense as a JSON object.
- **Error Responses:**
  - HTTP 400 for malformed JSON or invalid field values.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the wallet does not belong to the authenticated user.

5.5.14 Update Recurring Expense

Updates an existing recurring expense. Ownership is verified through the expense's parent wallet.

- **URL:** `/api/expenses/{expenseId}`
- **Method:** `PUT`
- **URL Parameters:**
  - `expenseId` in path (UUID)

- **Data Parameters (JSON body):**
  - `label` = {string} (required)
  - `amount` = {string} (required)
  - `periodicity` = {string} (required)
  - `nextDue` = {string} (required, format YYYY-MM-DD)
- **Success Response:**
  - HTTP 200 with the updated recurring expense as a JSON object.
- **Error Responses:**
  - HTTP 400 for malformed JSON, invalid UUID, or invalid field values.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the expense's wallet does not belong to the authenticated user.
  - HTTP 404 if the recurring expense does not exist.

5.5.15 Delete Recurring Expense

Deletes an existing recurring expense. Ownership is verified.

- **URL:** `/api/expenses/{expenseId}`
- **Method:** `DELETE`
- **URL Parameters:**
  - `expenseId` in path (UUID)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 204 No Content.
- **Error Responses:**
  - HTTP 400 if the expense ID is missing or invalid.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the expense's wallet does not belong to the authenticated user.
  - HTTP 404 if the recurring expense does not exist.

5.5.16 Toggle Recurring Expense Active Status

Toggles a recurring expense between active and inactive. Ownership is verified.

- **URL:** `/api/expenses/{expenseId}/toggle`
- **Method:** `POST`
- **URL Parameters:**
  - `expenseId` in path (UUID)

- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with the updated recurring expense as a JSON object.
- **Error Responses:**
  - HTTP 400 if the expense ID is invalid.
  - HTTP 401 if the user is not authenticated.
  - HTTP 403 if the expense's wallet does not belong to the authenticated user.
  - HTTP 404 if the recurring expense does not exist.

5.5.17 List PAC Plans

Returns all PAC plans belonging to the authenticated user, forwarded to a JSP page.

- **URL:** `/pac`
- **Method:** `GET`
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with the PAC plan list page rendered via JSP.
- **Error Responses:**
  - Redirect to `/login` if unauthenticated.

5.5.18 Create PAC Plan

Creates a new PAC (DCA) investment plan for the authenticated user.

- **URL:** `/pac/create`
- **Method:** `POST`
- **Data Parameters (form):**
  - `assetName` = {string} (max 120 characters)
  - `monthlyAmount` = {string} (positive numeric value)
  - `targetAmount` = {string} (positive numeric value)
  - `startDate` = {string} (format YYYY-MM-DD)
- **Success Response:**
  - Redirect to `/pac?success=created`.
- **Error Responses:**
  - Redirect to `/pac?error=validation` for invalid input.
  - Redirect to `/pac?error=server_error` on internal failure.

5.5.19 Edit PAC Plan

Updates an existing PAC plan. Ownership is verified.

- **URL:** `/pac/edit`
- **Method:** `POST`
- **Data Parameters (form):**
  - `planId` = {UUID}
  - `assetName` = {string}
  - `monthlyAmount` = {string}
  - `targetAmount` = {string}
  - `startDate` = {string}
  - `active` = {string} ("true" or "false")
- **Success Response:**
  - Redirect to `/pac?success=updated`.
- **Error Responses:**
  - Redirect to `/pac?error=validation` for invalid input or malformed UUID.
  - Redirect to `/pac?error=not_found` if plan does not exist.
  - Redirect to `/pac?error=unauthorized` if plan belongs to another user.

5.5.20 Delete PAC Plan

Deletes an existing PAC plan. Ownership is verified.

- **URL:** `/pac/delete`
- **Method:** `POST`
- **Data Parameters (form):**
  - `planId` = {UUID}
- **Success Response:**
  - Redirect to `/pac?success=deleted`.
- **Error Responses:**
  - Redirect to `/pac?error=validation` for missing or malformed UUID.
  - Redirect to `/pac?error=not_found` if plan does not exist.
  - Redirect to `/pac?error=unauthorized` if plan belongs to another user.

5.5.21 Compute Forecast

Computes a 24-month financial projection for the authenticated user under the specified scenario and stores the result as JSONB in the `forecasts` table. The projection is based on: current wallet balances, the 3-month average of income and expenses (with adaptive windowing for newer users), active recurring expenses normalized to monthly equivalents, and active PAC plan contributions. All wallets must use the same currency.

- **URL:** `/api/forecast?scenario={scenario}`
- **Method:** `POST`
- **URL Parameters:**
  - `scenario` = {string} (required, one of: `OPTIMISTIC`, `CONSERVATIVE`, `STRESS_TEST`)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 201 with a JSON object containing:
    - `id`: the forecast UUID
    - `scenario`: the scenario name
    - `projection`: a JSON array of 24 monthly entries, each containing `month`, `income`, `expense`, `pacContribution`, `netFlow`, `balance`, and `currency`
    - `createdAt`: the computation timestamp
- **Error Responses:**
  - HTTP 400 if the scenario parameter is missing or invalid.
  - HTTP 400 if wallets use mixed currencies.
  - HTTP 401 if the user is not authenticated.

5.5.22 Retrieve Latest Forecast

Retrieves the most recently computed forecast for the authenticated user and the specified scenario.

- **URL:** `/api/forecast?scenario={scenario}`
- **Method:** `GET`
- **URL Parameters:**
  - `scenario` = {string} (required, one of: `OPTIMISTIC`, `CONSERVATIVE`, `STRESS_TEST`)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with the forecast JSON object (same structure as the compute response).
- **Error Responses:**
  - HTTP 400 if the scenario parameter is missing or invalid.
  - HTTP 401 if the user is not authenticated.
  - HTTP 404 if no forecast has been computed yet for the given scenario.

5.5.23 Guest Report Page

Displays the guest calculator page with options for loan amortization, compound interest, and budget analysis.

- **URL:** `/guest/report`
- **Method:** `GET`
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with the calculator page rendered via JSP.

5.5.24 Compute Loan Amortization

Calculates a loan amortization schedule and stores the result in the guest session's report data.

- **URL:** `/report/loan`
- **Method:** `POST`
- **Data Parameters (form):**
  - `principal` = {number} (loan amount)
  - `annualRate` = {number} (annual interest rate)
  - `months` = {integer} (repayment period in months)
- **Success Response:**
  - Redirect to `/guest/report` with results stored in the guest session.
- **Error Responses:**
  - Redirect to `/guest/report?error=validation` for invalid input.

5.5.25 Compute Compound Interest

Calculates a compound interest projection and stores the result in the guest session's report data.

- **URL:** `/report/compound`
- **Method:** `POST`
- **Data Parameters (form):**
  - `initialCapital` = {number} (starting amount)
  - `monthlyContribution` = {number} (monthly deposit)
  - `annualRate` = {number} (annual interest rate)
  - `years` = {integer} (investment period in years)
- **Success Response:**
  - Redirect to `/guest/report` with results stored in the guest session.
- **Error Responses:**
  - Redirect to `/guest/report?error=validation` for invalid input.

5.5.26 Compute Budget Breakdown

Calculates income vs. expense totals with categorized breakdowns and stores the result in the guest session.

- **URL:** `/report/budget`
- **Method:** `POST`
- **Data Parameters (form):**
  - `incomeCategory[]` = {string[]} (array of income category names)
  - `incomeAmount[]` = {number[]} (array of income amounts)
  - `expenseCategory[]` = {string[]} (array of expense category names)
  - `expenseAmount[]` = {number[]} (array of expense amounts)
- **Success Response:**
  - Redirect to `/guest/report` with results stored in the guest session.
- **Error Responses:**
  - Redirect to `/guest/report?error=validation` for invalid input.

5.5.27 Export Report as PDF

Exports the accumulated guest report data as a downloadable PDF file.

- **URL:** `/report/export?token={sessionToken}`
- **Method:** `GET`
- **URL Parameters:**
  - `token` = {UUID} (the guest session token)
- **Data Parameters:** None
- **Success Response:**
  - HTTP 200 with `Content-Type: application/pdf` and the PDF file as the response body.
- **Error Responses:**
  - Redirect to `/guest/report?error=no_data` if no report data exists.
  - HTTP 403 if the token does not match the current session.

6 Group Members Contribution

Alessandro Perin Architected and implemented the core Data Logic Layer, including the PostgreSQL database schema (`init.sql`) with advanced functionalities such as UUIDs, ENUMs, JSONB, and defensive DDL constraints. Developed the fundamental DAO layer (`WalletDAO`, `TransactionDAO`, `RecurringExpenseDAO`) and the comprehensive REST API backend for the Recurring Expenses feature. This included building the Service logic with strict ownership validation, custom exception handling, and an extensive suite of integration tests. Finally, drafted the LaTeX documentation relative to the Data Logic Layer and the Entity-Relationship structures.

Rajmonda Bardhi Designed and implemented the Dashboard Aggregation Service. Built the `DashboardDAO` with optimized SQL queries to compute per-wallet balances and aggregate statistics (total equity, year-to-date expenses, and year-to-date savings) grouped by currency. Developed the `DashboardService` to handle data retrieval and create the `DashboardSummary` response object. Created the `DashboardServlet` handling authenticated GET requests, and the corresponding JSP view. Defined the model classes `DashboardStats`, `DashboardSummary`, and `WalletSummary`. Added additional query methods to `TransactionDAO` and wrote unit and integration tests for the dashboard layer. Finally, wrote the Objectives section of the LaTeX documentation, describing the system goals and the rationale behind each feature.

Meviola Karriqi Collaborated with the team on the initial project design on the Miro board, which included the first draft of the database schema (later refined by the team). Wrote the Trello tickets and helped divide the work across team members throughout the project. On the implementation side, developed the Wallets feature (Servlet/JSP with PRG), the Transactions REST/JSON API, and the PAC plans feature. Added `java.util.logging` across the servlets and the `AuthenticationFilter` to trace errors and security-relevant events. Wrote the Main Functionalities, REST API Summary, REST API Details, and REST Error Codes sections of this LaTeX documentation.

Danil Smirnov Designed and implemented the user authentication flow, with particular focus on the registration and login functionalities. Developed the server-side logic for account creation and credential validation through components such as `LoginServlet`, `RegisterServlet`, `UserService`, and `UserDAO`, and connected the login and registration pages to the backend workflow. Defined the initial domain model classes used across the application, including `User`, `Wallet`, and `Transaction`. Also contributed to the initial setup of page access control filters, such as `AuthenticationFilter` and `GuestSessionFilter`, to separate protected pages from guest-only pages.

Giacomo Masiero